

ROMAN URDU STUDY NOTES

CSS Fundamentals Se Tailwind Tak

Box model, Flexbox, positioning, responsive design, Google Fonts, aur Tailwind CSS — sab kuch practical project (Chris Courses hero + stats section) ke sath explain kiya gaya hai.

Topics

Display • Flexbox • Position • Media Queries • TailwindCSS Course Video TranscriptRoman Urdu + Code Examples

Based On

Style

CSS Kya Hai?

CSS (Cascading Style Sheet) ek coding language hai jo HTML ko style karne ke liye use hoti hai — yani jo bhi HTML page banaya hai usko "pretty" banana. CSS hamesha HTML ke sath tandem mein use hoti hai — pehle HTML file honi chahiye, phir uske andar `<style>` tag likh kar CSS shuru karte hain.

Class Banane Ka Tareeqa

CSS class ek **dot (.)** se start hoti hai, phir koi bhi naam, phir curly brackets `{ }` jinke andar properties likhi jati hain.

```
.text-red {  
  color: red; /* property : value ; */  
}
```

- **Property** — kya cheez style karni hai (jaise `color`)
- **Value** — kis tarah style karni hai (jaise `red`)
- Colon `:` property aur value ko separate karta hai
- Semicolon `;` statement complete hone ka signal deta hai

Class ko HTML element par apply karne ke liye `class="text-red"` attribute lagate hain. Ek element par multiple classes bhi laga sakte ho — koi limit nahi.

Tip: Ek CSS class mein jitni chahe properties daal sakte ho, aur poori site par jitni chahe classes bana sakte ho — mix-and-match karke exact look milta hai.

Project Ka Setup

Naya project shuru karne ke liye sirf ek file chahiye: `index.html` . Isko kisi bhi folder mein save karo (jaise Web/css-course/), phir double-click karke Chrome mein open karo. Har save ke baad refresh karke changes dekh sakte ho.

Figma Se Design Lena

Real-world client projects mein design pehle se **Figma** jaisi tool mein banaya hota hai. Developer ka kaam hota hai us design ko HTML/CSS mein replicate karna.

Kyun Figma?

Kisi bhi element par click karke uski exact properties (font family, size, weight, color) sidebar mein dikh jati hain — guessing ki zaroorat nahi.

Desktop + Mobile Design

Hamesha dono design maango — warna mobile site "garbage" lagegi. Developer hi responsible hota hai mobile responsiveness ke liye.

HTML Markup Likhne Ka Order

Design ko hamesha **top se bottom** markup karo:

1. **Header tag** — logo se sign-up button tak
2. **Div tags** — generic grouping ke liye (divider samajh lo)
3. **Anchor tags (<a>)** — links ke liye
4. **Button element** — CTA buttons ke liye

```
<header>
  <div>
    <img> <!-- logo -->
    <a>Pricing</a>
    <a>Courses</a>
  </div>
  <div>
    <a>Login</a>
    <button>Sign Up</button>
  </div>
</header>
```

Golden Rule: Sirf ek generic grouping ho to `<div>` use karo. Meaningful structure ke liye semantic tags (`header` , `section` , `nav`) use karo.

Background Color & Hexadecimal

background-color Property

```
.bg-gray-900 {  
  background-color: #12182b;  
}
```

Color 3 tareeqon se specify ho sakta hai: **named colors** (red, blue), **hexadecimal** (#ff0000), ya **rgba/hsl** values. Designers usually Figma se **hex code** dete hain.

Hexadecimal Format Samjho

Hex code 6 digits ka hota hai: **#RRGGBB** — har digit 0 se 9, ya a se f tak ho sakta hai.

Digits	Represent	Range
Pehle 2	Red	00 (dark) → ff (bright)
Beech ke 2	Green	00 (dark) → ff (bright)
Aakhri 2	Blue	00 (dark) → ff (bright)

```
background-color: #ff0000; /* pure red */  
background-color: #00ff00; /* pure green */  
background-color: #0000ff; /* pure blue */
```

Text Color

```
.text-white {  
  color: white;  
}
```

Dark background par default text (dark purple/black) legible nahi hota — **color: white** use karke contrast fix karte hain.

Design Tip: Har color ke liye ek "palette" banao (100 se 900 tak shades). Jitni zyada shades honge, utna better contrast aur focus control milega.

Browser Default Styling Fix Karna

Body Ka Default Margin

Browser by default `<body>` tag ko **8px margin** deta hai — isi wajah se page ke edges par unwanted spacing dikhti hai. Element ko **Inspect** karke (right-click → Inspect) is orange highlight (margin) ko dekh sakte ho.

```
body {  
  margin: 0; /* browser default margin hata do */  
}
```

Yaad rahe: Element ko select karke `.class` use karte hain, lekin body/html/head jaise auto-generated tags ko **directly tag name** se select karte hain — dot ki zaroorat nahi.

Inspect Element Ka Use

- **Orange border** = margin (element ke bahar ki spacing)
- **Green border** = padding (element ke andar ki spacing)
- Browser ke **Elements** tab mein live HTML/CSS dekh aur edit kar sakte ho (temporary, refresh par reset ho jata hai)
- **Computed** tab mein kisi bhi property ki default value dekh sakte ho

Pro Tip: Browser console mein directly CSS test karna fast hai, lekin permanent karne ke liye code copy karke apni HTML file mein paste karna zaroori hai.

Display Property

Display property decide karti hai ke element browser mein kaise layout hota hai. 5 main values hain:

Value	Behaviour
<code>block</code>	Parent container ki poori width le leta hai; naya element neeche push ho jata hai (div ka default)
<code>inline</code>	Sirf content jitni space leta hai; margin top/bottom kaam nahi karta (a tag ka default)
<code>inline-block</code>	Inline jaisa hi, magar margin/padding top-bottom bhi kaam karta hai
<code>none</code>	Element document flow se completely hat jata hai — invisible (popups, dropdowns ke liye)
<code>flex</code>	Children ko side-by-side layout karta hai + advanced alignment control deta hai

```
.block { display: block; }
.inline-block { display: inline-block; }
.inline { display: inline; }
.hidden { display: none; }
.flex { display: flex; }
```

Har HTML tag ka default display hota hai: `div` = block, `a` = inline. Computed tab mein check kar sakte ho.

Golden Rule: `display: flex` hamesha **parent** container par lagao — child elements par nahi. Flex parent apne saare children ko automatically affect karta hai, chahe wo pehle kuch bhi ho.

Flexbox Deep Dive

Flexbox professional development mein sabse zyada use hone wala layout tool hai. Sirf 3 classes se 90% kaam ho jata hai:

1. justify-content — Horizontal Alignment

```
.justify-between { justify-content: space-between; }  
.justify-center { justify-content: center; }
```

Children ko horizontal axis (left→right) par control karta hai. **space-between** pehla element far-left aur last element far-right rakhta hai, baaki equal spacing.

2. align-items — Vertical Alignment

```
.items-center { align-items: center; }
```

Children ko vertical axis par center karta hai (jaise logo aur nav links ko same line par align karna).

3. Nested Flex Containers

Flex classes ko multiple parent divs par "stack" kar sakte ho — jaise header ke andar do groups (logo+links, login+signup) ko phir header par bhi flex laga kar side-by-side kar sakte ho.

```
<header class="flex justify-between items-center">  
  <div class="flex items-center"> // logo + nav links  
  <div class="flex items-center"> // login + signup  
</header>
```

Reference: [css-tricks.com](https://css-tricks.com/a-complete-guide-to-flexbox/) ka "A Complete Guide to Flexbox" — sab justify-content aur align-items options ka visual guide milta hai (Flex-start, Flex-end, Center, Stretch, waghera).

Margin vs Padding

Margin

Element ke **bahar** ki spacing. Ek element ko doosre element se door karne ke liye use hota hai.

Padding

Element ke **andar** ki spacing (content aur background ke beech). Background color ke sath spacing chahiye ho to padding use karo.

Decision Rule: Agar background color hai aur content se gap chahiye → **padding**. Agar do alag elements (jaise 2 headers) ke beech gap chahiye → **margin**.

Individual Side Properties

```
padding-top: 18px;
padding-bottom: 18px;
padding-left: 12px;
padding-right: 12px;
```

Shorthand Syntax

Same syntax margin ke liye bhi kaam karta hai:

```
padding: 18px; /* sab sides same */
padding: 18px 12px; /* top-bottom 18px, left-right 12px */
padding: 18px 12px 100px 80px; /* top right bottom left (clockwise) */
```

Utility Classes (Atomic Approach)

```
.py-8 { padding-top: 8px; padding-bottom: 8px; }
.px-16 { padding-left: 16px; padding-right: 16px; }
```

CSS Cascade Warning: Jo class CSS mein **neeche** likhi ho wo upar wali ko override kar deti hai — chahe HTML mein order kuch bhi ho. `.px-16` ke andar agar galti se `padding: 0 16px` likha (yani top/bottom bhi zero force kiya) to wo `.py-8` ko override kar dega. Isliye sirf specific properties set karo (padding-left/right), poora shorthand nahi.

Inline Styles — Bad Nahi Hain!

Ek unique, one-time style ke liye `style="padding: 18px 12px"` use karna theek hai — separation of concerns ka "old school" rule modern development mein utna zaroori nahi jitna pehle tha. Reusable styles ke liye class banao, unique styles ke liye inline theek hai.

Logo, Images & SVG

SVG vs PNG/JPEG

Format	Kab Use Karein
SVG	Icons, logos, graphics — vectorized, kabhi bhi pixelate nahi hota, size badhao to bhi crystal clear rehta hai
PNG / JPEG	Photos, complex images jinme bohat colors ho

```

```

Figma se logo lene ka tareeqa: element select karo → right-click → Copy as → Copy as SVG. Isko seedha HTML mein paste **na karo** (code messy ho jata hai) — nayi **.svg** file banao aur usko reference karo.

Relative Paths: **./img/logo.svg** ka matlab hai "isi directory ke andar img folder mein logo.svg". Files ko organize rakhne ke liye images ko alag **img/** folder mein rakho.

Width Control

```
.w-full { width: 100%; } /* parent container ki full width */
```

Image parent container se bahar overflow na ho, is liye width ko 100% set karna zaroori hai — warna page left-right scroll karne lagta hai.

Google Fonts Integrate Karna

Steps

1. **fonts.google.com** par jao, chahe wala font search karo (jaise "Lato")
2. Font style select karo (jaise Regular 400, Bold 700) — "Select this style"
3. Link tags copy karke `<head>` ke andar paste karo
4. CSS "specify families" rule copy karo aur apni class mein use karo

```
<!-- head ke andar -->
<link href="https://fonts.googleapis.com/..." rel="stylesheet">

/* CSS mein */
body {
  font-family: "Lato", sans-serif; /* fallback bhi rakho */
}
```

Best Practice: Font-family ko `body` tag par set karo, taake har child element automatically wahi font le — har element par alag se class lagane ki zaroorat nahi.

Multiple Weights (Regular + Bold)

Agar sirf "Regular 400" pull kiya aur `font-weight: bold` use kiya, to jin users ke computer par wo font install nahi hai unko bold nahi dikhega. Isliye Google Fonts se **font weights** (400 aur 700) select karo:

```
<link href="...family=Lato:wght@400;700" >
```

Font Anti-Aliasing (Crisp Text)

Browsers by default text ko smooth nahi karte. Ye 2 lines hamesha `body` par add karo:

```
body {
  -webkit-font-smoothing: antialiased;
  -moz-osx-font-smoothing: grayscale;
}
```

Text Decoration, Transform & Weight

Underline Hatana

```
.no-underline { text-decoration: none; }
```

Anchor tags (`<a>`) by default underlined hote hain. Header/nav links mein underline zaroori nahi (user ko already pata hai ye clickable hai).

Uppercase Text

```
.uppercase { text-transform: uppercase; }
```

CTA buttons (jaise "Sign Up") ko attention grab karne ke liye uppercase kiya jata hai.

Font Weight

```
.font-bold { font-weight: bold; } /* ya 700 */
```

Weight scale 100 (thin) se 900 (very bold) tak hota hai. Values: `bold` , `bolder` , `lighter` , `normal` , ya numeric.

Font Size & Letter Spacing

```
.text-sm { font-size: 14px; }  
letter-spacing: 1px; /* characters ke beech spacing */
```

Default Font Family Trap: Buttons ka default font family `Arial` hota hai — chahe body mein koi bhi font family set ho! Fix karne ke liye `button` tag ko explicitly target karo:

```
button { font-family: "Lato", sans-serif; }
```

Border & Border Radius

Border Hatana

Buttons ko default mein ek ugly "beveled" border milta hai — modern design ke liye hata do:

```
.border-none { border: none; }
```

Rounded Corners

```
.rounded-full { border-radius: 9999px; } /* pura pill-shape */  
border-radius: 4px; /* halka sa rounding */
```

Jitna zyada **border-radius** value, utna zyada curve. Bohat high value (jaise 9999px) hamesha fully-rounded pill shape deta hai, chahe padding kuch bhi ho.

Complete Button Recipe

```
.btn-primary {  
  background-color: #5e9eff;  
  color: white;  
  border: none;  
  border-radius: 9999px;  
  padding: 12px 28px;  
  font-weight: bold;  
  text-transform: uppercase;  
  font-size: 14px;  
  letter-spacing: 1px;  
}
```

Clickable Area Tip: Links ke beech spacing ke liye **margin** ki jagah **padding** use karo — isse whitespace bhi clickable ban jata hai, user experience better hota hai.

Position: Relative & Absolute

position: relative

```
.relative { position: relative; }
```

Element apni original position se **top** , **right** , **bottom** , **left** properties se move ho sakta hai, lekin document flow mein space still reserve rehti hai.

position: absolute

```
.absolute { position: absolute; }
```

Element document flow se **completely nikal jata hai** — dusre elements is par depend nahi karte. Ye apne **nearest positioned parent** (jis ka position relative/absolute ho) ke relative position hota hai.

Critical Rule: Agar koi parent element mein **position: relative** nahi hai, to absolute child seedha **browser window** ke relative position ho jayega — jo galat layout create karta hai. Hamesha parent-child pair banao.

Z-Index — Layering

```
.z-10 { z-index: 10; }
```

Z-axis (screen ke andar-bahar) positioning — **bada number** = closer to viewer (foreground), **chhota number** = background. Z-index sirf tab kaam karta hai jab element ka position **relative** ya **absolute** ho.

inset-0 Shortcut

```
.inset-0 {  
  top: 0; right: 0;  
  bottom: 0; left: 0;  
}
```

Ye 4 properties ek sath zero par set karke element ko parent ki **poori width/height** tak stretch kar deta hai — overlays banane ke liye perfect (jaise dark overlay video ke upar).

Background Video & Object-Fit

Video Tag Setup

```
<video
  loop autoplay playsinline muted
  src="./videos/galaxy.mp4"
></video>
```

Attribute	Kaam
<code>loop</code>	End hone par phir se shuru se play
<code>autoplay</code>	User click ke bagair khud play ho
<code>playsinline</code>	Mobile par inline play (fullscreen force nahi karta)
<code>muted</code>	Zaroori — bina mute autoplay kaafi browsers mein block ho jata hai

Absolute vs Relative URL: Kisi doosri site ka video seedha use na karo (absolute URL) — agar wo site apna video change/delete kare to tumhara site break ho jayega. Video download karke apni `videos/` folder mein rakho aur relative path use karo.

object-fit: cover

```
.object-cover { object-fit: cover; }
```

Video/image ko parent container ke **exact height aur width** tak fill karta hai — chahe screen size kuch bhi ho — aur **aspect ratio maintain** karta hai (zoom karke, stretch nahi karke). Bina isko blank spacing dikhti hai jab screen resize hoti hai.

Containers & Max-Width

Bade screens (jaise 4K monitor) par agar content full-width le le, to text bohat dur left/right chala jata hai jo hard-to-read banata hai. Solution: **container pattern**.

Max-Width

```
max-width: 1200px; /* is se zyada width nahi le sakta */
```

max-width aur set **width** mein farq: max-width chhoti screens par scale down ho sakta hai, lekin large screens par ek limit se zyada nahi grow karega.

Auto-Centering

```
.mx-auto {  
  margin-left: auto;  
  margin-right: auto;  
}
```

margin: auto browser ko automatically equal margin left aur right calculate karne deta hai — element perfectly center ho jata hai.

```
<div class="container mx-auto" >  
  <!-- max-width: 1200px + auto centering -->  
</div>
```

Important: Container class ko un elements par mat lagao jinhe full-width chahiye (jaise background video) — sirf text/content wrapper divs par lagao, warna video bhi 1200px tak limit ho jayegi.

Media Queries & Viewport

Viewport Meta Tag — Sabse Zaroori

```
<meta name="viewport" content="width=device-width, initial-scale=1.0" >
```

Ye tag har project mein hamesha daalo! Bina is ke, Chrome DevTools ka mobile emulator (aur real phones) sahi tarah scale nahi karte — sab kuch galat dimensions mein render hoga.

Media Query Syntax

```
/* Sab kuch bahar = mobile styling (default) */

@media (min-width: 640px) {
  /* Ye sab sirf 640px se badi screens par chalega */
  .bg-blue { background-color: blue; }
}
```

Common Breakpoints

Naam	Min-Width	Device
sm	640px	Large phones
md	768px	Tablets
lg	1024px	Small desktops
xl	1280px	Desktops
2xl	1536px	Large desktops

Mobile-First Approach

Sab default styles **mobile ke liye** likho, aur media query ke andar sirf wo cheezein overwrite karo jo desktop par different chahiye (jaise bada font-size, extra padding, navigation dikhana).

Emulator Tip: Chrome DevTools mein "Toggle Device Toolbar" icon se different screen sizes emulate kar sakte ho. Agar viewport tag add karne ke baad emulator sahi nahi dikh raha, to emulator ko band karke دوبारा open karo.

Hide/Show Elements Par Screen Size

Chhoti screens par bohat sare nav links fit nahi hote — solution hai unhe hide karna aur hamburger icon dikhana.

```
/* Mobile: nav hidden, hamburger visible */
nav { display: none; }
.hamburger { display: block; }

/* Tablet+ (768px se upar): nav visible, hamburger hidden */
@media (min-width: 768px) {
  nav { display: block; }
  .hamburger { display: none; }
}
```

SVG Icons — Hero Icons

heroicons.com se free SVG icons directly copy-paste kar sakte ho. SVG ka color set karne ke liye:

```
.hamburger-icon { color: white; } /* stroke: currentColor use karta hai */
```

SVG mein `stroke="currentColor"` ho to wo automatically parent ke `color` property se apna color leta hai.

Flex + Hidden Interaction: Agar koi element `display: none` se hidden hai, to Flexbox use bilkul ignore kar deta hai jaise wo exist hi nahi karta — space allocation mein include nahi hota.

Tailwind CSS — Introduction

Tailwind CSS ek utility-first CSS framework hai jo pre-built classes deta hai (jaise `flex` , `pt-4` , `text-center`) — jo basically wahi classes hain jo hum manually likh rahe the, lekin ready-made.

CDN Se Install Karna (Beginner-Friendly)

```
<!-- head ke andar, jitni jaldi ho sake -->
<script src="https://cdn.tailwindcss.com"></script>
```

Production ke liye recommended nahi (Tailwind khud batata hai), lekin seekhne aur prototyping ke liye sabse easy tareeqa hai.

Browser Reset Ka Asar

Tailwind import karte hi ek **reset stylesheet** automatically apply hoti hai — sab HTML elements ki default styling (jaise H1 ka bold, ya default line-height) zero ho jati hai.

- H1 ab bold nahi rahega → manually `font-bold` lagana padega
- Line-height change ho sakti hai → text ka spacing tight ho sakta hai
- Apne banaye hue duplicate classes hata do (jaise apna khud ka `.flex` , `.hidden`) — Tailwind pehle se deta hai

Line-Height Fix

```
.leading-none { line-height: 1; } /* font-size ke barabar */
.leading-tight { line-height: 1.25; }
.leading-loose { line-height: 2; }
```

Search Karna Seekho: tailwindcss.com par direct search bar se koi bhi CSS property search karo (jaise "font-bold" ya "opacity") — Tailwind exact class name dikha deta hai. Apni khud ki class likhne se pehle hamesha check karo Tailwind mein already available to nahi.

Tailwind Responsive Prefixes

Tailwind manually media-query likhne ki zaroorat khatam kar deta hai — bas class name ke aage breakpoint prefix laga do.

Prefix	Min-Width
sm:	640px
md:	768px
lg:	1024px
xl:	1280px
2xl:	1536px

Escape Character

Colon `:` CSS mein special meaning rakhta hai, isliye jab manually class banate ho to backslash se "escape" karna padta hai:

```
.sm\:text-60px { font-size: 60px; }  
/* HTML mein use: class="sm:text-60px" (backslash nahi likhte) */
```

Chaining Breakpoints

```
<a class="hidden sm:inline lg:hidden">Pricing</a>
```

Ye example: mobile par hidden → 640px se dikhna shuru → 1024px par phir hidden ho jaye (jab bade screen par bhi hamburger chahiye ho).

Practical Example — Nav Hide/Show

```
<nav class="hidden md:block"> ... </nav>  
<svg class="md:hidden text-white"> ... </svg> <!-- hamburger -->
```

EM vs REM Units

em unit

Parent element ke font-size ke relative. Agar parent 16px hai aur child **1.25em** hai, to child = 20px. Parent change ho to child bhi change hoga (nested confusion ho sakta hai).

rem unit

Sirf root **<html>** tag ke font-size ke relative (default 16px).
Consistent aur predictable — **zyada recommended**.

```
/* html { font-size: 16px } default hota hai */  
font-size: 1.25rem; /* = 20px */  
margin-top: 1rem; /* = 16px */
```

Tailwind Ka Spacing Scale

Tailwind apna standardized rem-based scale deta hai — font-size, margin, padding sab isi scale se follow karte hain:

Class Suffix	REM	Pixels
1	0.25rem	4px
2	0.5rem	8px
4	1rem	16px
8	2rem	32px
16	4rem	64px
32	8rem	128px

```
<div class="mt-4 px-6 pt-32"> ... </div>
```

Fayda: Ek consistent scale use karne se pura site "harmonious" lagta hai — random 17px, 23px jaisi values se bachte hain.

Opacity, Overlays & Gradients

Opacity

```
.opacity-50 { opacity: 0.5; } /* 0 = invisible, 1 = fully visible */
```

Background video par text legible banane ke liye ek **black overlay div** (position: absolute + inset-0) daalte hain jiski opacity 50% hoti hai.

Linear Gradient

```
.bg-gradient-overlay {  
  background-image: linear-gradient(to bottom, black, transparent);  
}
```

Solid overlay ki jagah gradient zyada natural lagta hai — top par dark (black), neeche fade hoke transparent. Text jahan hai wahan darkness zyada, baaki jagah kam.

```
<div class="absolute inset-0 bg-gradient-to-b from-black to-transparent"></div>
```

Overlay Ka Height Control

Agar gradient/overlay ko default se zyada tak fade karna hai, to **inset-0** ke bottom value ko negative karke div ko taller bana do (jaise **bottom: -35px** inline style se override).

Responsive Consideration: Overlay/gradient sirf desktop par jaruri ho to **hidden md:block** use karo — mobile par simpler black overlay hi kaafi hai.

Divide Utility & Width Fractions

Equal-Width Flex Children

```
.w-1\3 { width: 33.333%; }
```

3 stat boxes (jaise "500+ Lessons", "100+ Hours", "5000+ Students") ko equal width dene ke liye percentage-based width use karo — static pixels nahi, taake responsive resize par sab proportion maintain rahe.

Divide-X — Automatic Dividers

```
<div class="flex divide-x divide-gray-600">
  <div class="w-1/3">Box 1</div>
  <div class="w-1/3">Box 2</div>
  <div class="w-1/3">Box 3</div>
</div>
```

divide-x automatically har child ke **beech** mein ek vertical border laga deta hai (edges par nahi) — manually border-left/right lagane se better tareeqa.

Manual Borders (Alternative)

```
.border-b { border-bottom-width: 1px; }
.border-solid { border-style: solid; }
.border-gray-500 { border-color: #6b7280; }
```

3 Cheezein Zaroori: Border dikhne ke liye width + style + color teeno chahiye hote hain — sirf color set karne se kuch nahi hoga.

Quick Cheat Sheet

Category	Classes
Display	block, inline, inline-block, hidden, flex
Flex	justify-between, justify-center, items-center
Spacing	p-*, px-*, py-*, m-*, mx-auto, mt-*
Position	relative, absolute, inset-0, z-10 → z-50
Sizing	w-full, w-1/3, h-full, max-w-*
Typography	text-white, font-bold, uppercase, no-underline, leading-none
Border	border-none, rounded-full, border-b, divide-x
Effects	opacity-50, bg-gradient-to-b, from-black, to-transparent
Responsive	sm:, md:, lg:, xl:, 2xl:

Poore Workflow Ka Summary

1. Figma design se properties nikaalo (font, color, spacing, width)
2. HTML markup top-to-bottom likho (semantic tags use karo)
3. Mobile-first CSS likho, phir media queries se desktop enhance karo
4. Container + max-width + mx-auto pattern se large screens control karo
5. Repetitive inline styles ko utility classes mein convert karo
6. Jab pattern clear ho jaye, Tailwind CSS import karke development speed badhao

Next Steps: Popular Courses section, Features, Testimonials slider, Pricing, FAQ, aur Footer — same principles (Flexbox + Position + Tailwind) se develop honge.

NEW SECTION

CSS by web.dev

Google ke web.dev "Learn CSS" course se deeper, theoretical fundamentals — Box Model, Selectors, Nesting, Cascade aur Specificity. Ye woh cheezein hain jo samajh aa jayein to CSS ka "kyun" clear ho jata hai, sirf "kaise" nahi.

Source

Covers

Level

web.dev Learn CSS SeriesBox Model • Selectors • Nesting • Cascade • SpecificityIntermediate/Theory

Box Model — Foundation

CSS mein **har cheez ek box hai** — chahe wo sirf text ho, ya `border-radius` se circle jaisa dikh raha ho. Box model samajhna CSS ki sabse core foundation hai.

Classic Confusion: Agar `width: 100px` , `padding: 20px` , aur `border: 1px solid` set karo, to element **142px** wide ban jata hai — 100px nahi! Wajah: default box model mein padding aur border, width ke **upar add** hote hain.

Extrinsic vs Intrinsic Sizing

Extrinsic Sizing

Tum khud `width` / `height` set karte ho. Zyada control milta hai, lekin content overflow ho sakta hai agar box chhota pad jaye.

Intrinsic Sizing

Browser content ke hisab se khud size decide karta hai (default behaviour). Overflow ka chance kam, flexibility zyada.

`width: min-content` jaisi keywords use hoti hain.

Box Model Ke 4 Areas

Area	Kya Hai
Content Box	Jahan actual content (text/images) rehta hai — sabse variable area
Padding Box	Content ke around ka space, <code>padding</code> property se banta hai. Background yahan tak visible hota hai
Border Box	Padding ke around ka frame — <code>border</code> property se define hota hai
Margin Box	Sabse bahar ka space — <code>margin</code> se banta hai. <code>outline</code> aur <code>box-shadow</code> bhi yahan "paint" hote hain

Picture Frame Analogy: Content = artwork, Padding = white mounting board, Border = frame, Margin = space between frames on the wall.

Box Model — box-sizing Property

Har browser apni **user agent stylesheet** apply karta hai — default styling jo tumne khud CSS nahi likhi. Ye **display** aur **box-sizing** ke defaults set karti hai.

Default Display Values

```
div { display: block; }      /* full width leta hai */
span { display: inline; }    /* sirf content jitni width */
li { display: list-item; }
```

content-box (Default) vs border-box

```
/* DEFAULT — padding/border WIDTH mein ADD hote hain */
.my-box {
  width: 200px;
  border: 10px solid;
  padding: 20px;
}
/* Actual width = 200 + 40 (padding L+R) + 20 (border L+R) = 260px */
```

```
/* border-box — width/padding/border PUSH IN hote hain */
.my-box {
  box-sizing: border-box;
  width: 200px;
  border: 10px solid;
  padding: 20px;
}
/* Actual width = exactly 200px */
```

Universal Reset — Industry Standard

```
*,
*::before,
*::after {
  box-sizing: border-box;
}
```

Best Practice: Ye rule almost har modern CSS reset ka hissa hota hai — **border-box** zyada predictable behave karta hai. Isko apne har project ke top par daalo.

DevTools Se Debug Karna

Chrome DevTools mein element select karke **Computed** tab mein box model ka visual diagram milta hai — content, padding, border, margin sab layers dikhti hain with exact pixel values.

Selectors — Simple Selectors

CSS rule 2 hisson ka hota hai: **selector** (kis element ko target karna hai) aur **declarations** (property:value pairs). Simple selectors elements, classes, IDs, aur attributes ko target karte hain.

Selector	Symbol	Example	Matlab
Universal	*	* { color: hotpink; }	Har element ko match karta hai
Type	tag name	section { padding: 2em; }	Directly HTML tag ko target
Class	.	.my-class { color: red; }	class="my-class" wale sab elements
ID	#	#rad { border: 1px solid; }	Unique element (id sirf ek dafa use ho)
Attribute	[]	[data-type='primary']	Specific attribute/value wale elements

Attribute Selector Ke Operators

```
[href='example.com'] { } /* contains */
[href^='https']      { } /* starts with */
[href$='.com']        { } /* ends with */
[data-type='Primary' s] { } /* case-sensitive */
[data-type='primary' i] { } /* case-insensitive */
```

Grouping Selectors (Comma)

```
strong, em, .my-class, [lang] {
  color: red;
}
```

Comma se multiple selectors ko ek hi declaration block mein group kar sakte ho — repeat karne ki zaroorat nahi.

ID Selector Warning: ID selector bohat "sticky" hota hai (high specificity) — override karna mushkil ho jata hai. Isliye reusable styles ke liye classes better hain, IDs sirf unique one-off elements ke liye.

Pseudo-Classes & Pseudo-Elements

Pseudo-Classes — Single Colon (:)

Element ki **state** ke hisab se style apply karte hain — hover, focus, nth-child waghera.

```
a:hover {
  outline: 1px dotted green;
}

p:nth-child(even) {
  background: floralwhite;
}
```

Pseudo-Elements — Double Colon (::)

Jaise ek naya "element" insert kar rahe ho, bina extra HTML likhe.

```
.my-element::before {
  content: 'Prefix - '; /* start mein content insert */
}

li::marker {
  color: red; /* bullet points ka color */
}

::selection {
  background: black; /* highlighted text style */
  color: white;
}
```

Yaad Rakho: Single colon `:` = pseudo-**class** (state-based). Double colon `::` = pseudo-**element** (fake element insert karta hai). Purane browsers (IE8) ke liye backward-compatibility ki wajah se `:before` bhi kabhi kabhi accept hota hai, lekin `::before` hi correct modern syntax hai.

Complex Selectors — Combinators

Combinator wo character hota hai jo do selectors ke **beech** mein hota hai — parent-child ya sibling relationships define karta hai. **Yaad rahe:** CSS sirf neeche (children) ki taraf cascade kar sakta hai, upar (parent) ki taraf nahi.

Combinator	Symbol	Example	Matlab
Descendant	space	<code>p strong</code>	p ke andar kahin bhi strong (recursive, kitni bhi levels deep)
Child	<code>></code>	<code>.top > p</code>	Sirf direct children, nested nahi
Next Sibling	<code>+</code>	<code>h2 + p</code>	Turant baad wala sibling element
Subsequent Sibling	<code>~</code>	<code>h2 ~ p</code>	Koi bhi baad wala sibling (turant hona zaroori nahi)

Descendant vs Child Combinator — Farq

```
/* Descendant — RECURSIVE, sab levels affect */
.top div { padding-left: 1em; }

/* Child — SIRF direct children */
.top > div { padding-left: 1em; }
```

Common Bug: Agar "spacing between stacked elements" ke liye `.top * { margin-top: 1em; }` use kiya, to **saare** nested children bhi affect ho jayenge — extra unwanted spacing. Fix: `.top > * + *` use karo (sirf direct children, jo pehla nahi hai unko target karega).

Compound Selectors

```
a.my-class { color: red; } /* <a> jo my-class rakhta ho — dono sath */
```

Bina space ke selectors ko combine karna — specificity aur readability dono badhata hai.

CSS Nesting

Nesting stylesheets ko zyada organized aur maintainable banata hai — related rules ko group karke likh sakte ho.

```
/* Nested version */
.feature {
  button { color: blue; }
  .link { color: red; }
  .text { font-size: 1.3em; }
}

/* Equivalent flat version */
.feature button { color: blue; }
.feature .link { color: red; }
.feature .text { font-size: 1.3em; }
```

2-3 Levels Se Zyada Deep Mat Jao — bohat deep nesting maintain karna mushkil ho jata hai. Refactor karne ka socho agar zyada nested ho raha hai.

The & Selector — Parent Reference

Bina **&** ke, nested selector automatically **descendant** ban jata hai. Compound selector (jaise **.feature:last-child**) banane ke liye **&** zaroori hai.

```
.feature {
  &:last-child { } /* = .feature:last-child (compound) */
  & :last-child { } /* = .feature :last-child (descendant, space!) */
  &.highlight { } /* = .feature.highlight */
  & .highlight { } /* = .feature .highlight */
}
```

Chhota Sa Space Bada Farq Dalta Hai: **&.highlight** (no space) aur **& .highlight** (space) bilkul alag matlab rakhte hain — pehla compound selector hai, dusra descendant selector.

Nesting At-Rules

```
.feature {
  @media (min-width: 40em) { }
  @container (inline-size > 900px) { }
  @supports (display: grid) { }
  @layer component { h2 { } }
}
```

@media, **@container**, **@supports**, aur **@layer** sab nest ho sakte hain.

Invalid Nesting: **&** pseudo-elements (jaise **::before**) ko represent nahi kar sakta. Type selector ke sath compound banane ke liye type **pehle** aana chahiye bina space ke: **p&** valid hai, **&p** invalid hai.

The Cascade — Conflict Resolution

CSS = **Cascading** Style Sheets. Cascade ek algorithm hai jo decide karta hai jab multiple rules ek hi element ko target karein, to kaunsa jeetega. 4 stages hain, is order mein:

1. **Position & Order** — CSS kis order mein appear hoti hai
2. **Specificity** — kaunsa selector "strongest" hai
3. **Origin** — CSS kahan se aayi (browser default, user, ya tumhari authored CSS)
4. **Importance** — kuch rules zyada weight rakhte hain (jaise `!important`)

Order Matters: Sabse pehle Origin aur Importance check hoti hai, phir Specificity, aur sabse aakhir mein Position (sirf tab jab sab kuch tie ho).

Position & Order Of Appearance

```
.my-element {  
  background: green;  
  background: purple; /* ye jeetega - baad mein likha */  
}
```

Agar do declarations ki specificity/origin/importance **bilkul same** ho, to jo **baad mein** likhi ho wo jeetegi.

Origin — Sabse Kam Se Sabse Zyada Specific

#	Origin
1	User agent (browser default) styles
2	Local user styles (OS-level, extensions)
3	Authored CSS (tumhari likhi hui)
4	Authored <code>!important</code>
5	Local user <code>!important</code>
6	User agent <code>!important</code>

Importance — Rule Types Ki Weight

Kam se zyada important: **normal rules** → **animation** → `!important` → **transition**. Active transitions/animations, normal styles ko override kar dete hain kyunki unka expected behaviour hi visual state change karna hai.

DevTools Tip: Chrome DevTools "Styles" panel mein overwritten CSS **strikethrough** (crossed-out) dikhti hai — agar tumhari CSS bilkul dikh hi na rahi ho, to matlab wo selector match hi nahi kar rahi (typo check karo).

Specificity — ABC Scoring System

Specificity ek **triad** hai — **(A, B, C)** — decimal number nahi. Har selector type ek specific component ko increment karta hai.

Component	Represents	Selector Types
A	ID-like specificity	<code>#id</code>
B	Class-like specificity	<code>.class</code> , <code>[attr]</code> , <code>:hover</code>
C	Element-like specificity	<code>div</code> , <code>::before</code>

Comparison Rule

Pehle **A** compare hota hai, tie hone par **B**, phir tie hone par **C**. **(1,0,0)** hamesha **(0,4,3)** se zyada specific hai — chahe B/C kitne bhi bade hon.

```
*                /* (0,0,0) – universal selector, kuch add nahi karta */
div              /* (0,0,1) – type selector */
.my-class        /* (0,1,0) – class selector */
#myID            /* (1,0,0) – ID selector */
a.my-class.another-class[href]:hover /* (0,4,1) – cumulative! */
```

Specificity Cumulative Hoti Hai

Ek selector mein jitne bhi parts hon, sab add hote jate hain. Isliye lambi selector chains (jaise upar wali) bohat "sticky" ban jati hain — override karna mushkil.

Special Cases — :is(), :not(), :where()

Function	Specificity Behaviour
<code>:not()</code>	Khud kuch add nahi karta, lekin andar wale argument ki specificity count hoti hai
<code>:is()</code>	Sabse specific argument ki specificity leta hai
<code>:where()</code>	Hamesha (0,0,0) — chahe andar ID ho, phir bhi zero specificity

:where() Ka Fayda: Agar base styles ko `:where()` mein wrap karo, to koi bhi baad ki normal CSS unhe easily override kar sakti hai — bohat useful component libraries/design systems ke liye.

Factors Jo Specificity Ko Affect NAHI Karte

Inline Styles

`style="color:red"` specificity mein count nahi hoti — ye cascade ka **alag hi step** hai (specificity se pehle evaluate hoti hai). Override karne ke liye **!important** chahiye hota hai.

!important

!important specificity nahi badalta — origin change kar deta hai. Do **!important** declarations tie ho jayein to phir specificity dobara compare hoti hai.

Pragmatic Specificity Boost

```
.my-button.my-button { background: blue; } /* class repeat = (0,2,0) */
```

Caution: Agar baar baar specificity boost karni pad rahi hai, to ye sign hai ke selectors bohat specific likhe ja rahe hain. Refactor karne ka socho instead of stacking classes.

Golden Rule: Specificity ko sirf utna hi high rakho jitni **zaroorat** hai — maximum possible specificity ka peecha mat karo, warna future mein genuinely important CSS ko override karna mushkil ho jayega.

Inheritance

Kuch CSS properties automatically **inherit** ho jati hain agar child element par explicitly value set na ki ho. Inheritance sirf **neeche** ki taraf flow karta hai — parent se child, kabhi upar nahi.

```
html { color: lightslategray; } /* sab descendants ko milega */
body { font-size: 1.2em; }      /* html ka font-size affect nahi hota */
p    { font-style: italic; }     /* sirf p aur uske descendants */
```

Kaunsi Properties Default Mein Inherit Hoti Hain?

Mostly **text-related** properties: **color** , **font-family** , **font-size** , **font-weight** , **line-height** , **letter-spacing** , **text-align** , **text-transform** , **cursor** , **visibility** , **list-style** waghera.

Inherit NAHI Hoti: **margin** , **padding** , **border** , **background** , **width** / **height** , **display** , **position** — box-model aur layout properties generally inherit nahi hoti.

Control Karne Wale Keywords

Keyword	Kya Karta Hai
inherit	Force karta hai parent ki computed value lena — chahe property normally inherit na hoti ho
initial	Property ko uski CSS spec default value par reset karta hai
unset	Agar property naturally inherit hoti hai → inherit jaisa behave karta hai. Nahi to → initial jaisa
revert	Sirf current origin (tumhari authored CSS) ki styles undo karta hai, user-agent default par wapis
revert-layer	Sirf current cascade layer ki styles undo karta hai

```
.my-component strong { font-weight: inherit; } /* parent .my-component ka weight le lega */
aside strong         { font-weight: initial; } /* CSS default (normal) par reset */
aside p              { all: unset; }          /* future mein bhi hamesha unset rahega */
```

unset Ka Fayda: Har property yaad rakhna ke wo inherit hoti hai ya nahi mushkil hai — **unset** use karo aur browser khud sahi behavior figure kar lega.

Color — Choosing Colors

Type	Syntax	Note
Named	<code>color: hotpink;</code>	148 named colors available (red, tomato, goldenrod, waghera)
Hex	<code>#b71540</code>	Sabse popular syntax. RRGGBB format, 0-9 aur A-F range
RGB	<code>rgb(183 21 64)</code>	Red/Green/Blue 0-255 ya percentage mein
HSL	<code>hsl(344 79% 40%)</code>	Hue (0-360°) / Saturation % / Lightness %

Hex Alpha (Transparency)

```
#00000000 /* 0% alpha — fully transparent */
#00000080 /* 50% alpha */
#000000BF /* 75% alpha */
```

8-digit hex mein aakhri 2 digits alpha define karte hain. 3-digit shorthand bhi chalta hai: `#a4e` = `#aa44ee` .

RGB aur HSL Mein Alpha

```
color: rgb(0 0 0 / 50%); /* ya rgba(0,0,0,0.5) */
color: hsl(0 0% 0% / 50%); /* ya hsla(0,0%,0%,0.5) */
```

HSL Ka Color Wheel

- **Hue** — 0° se 360°, red se shuru (0 = 360 = red), color wheel ka origin
- **Saturation** — 0% = grayscale, 100% = fully vibrant
- **Lightness** — 0% = black, 100% = white, 50% = pure hue

HSL Ka Fayda: Predictable manipulation — agar tumhe sirf color ko lighter/darker karna hai, sirf Lightness change karo. Hue/Saturation same rehte hain.

Color — HD Colors & Manipulation

High Definition Color Functions

Function	Example	Use Case
<code>color()</code>	<code>color(srgb 0.9 0.2 0.4)</code>	Specific color space select karo, values 0-1 range mein
Display P3	<code>color(display-p3 1 .5 0)</code>	sRGB se 50% zyada colors — newer monitors ke liye
Oklab	<code>oklab(75% 0.1 0.1)</code>	Lightness, a (green-red), b (blue-yellow) — smooth gradients
OkLCh	<code>oklch(80% 0.1 200)</code>	Lightness, Chroma, Hue — perceptually uniform hue changes

color-mix() — Do Colors Blend Karna

```
background: color-mix(in oklch, blue 70%, white);
```

Color space aur interpolation direction (`shorter` / `longer`) specify kar sakte ho hue-based spaces ke liye. Lighter/darker variants banane ka aasan tareeqa — white/black ke sath mix karo.

Relative Color Syntax (RCS)

```
color: oklch(from red l c h); /* red ko as-is */
color: oklch(from var(--primary) calc(l * 0.9) c h); /* 10% darker */
```

Kisi bhi existing color se calculations karke naya color banate ho — `from` keyword ke baad channels (l, c, h) ko letters ki tarah use kar sakte ho, phir `calc()` se adjust karo.

System Colors & Keywords

```
background: transparent; /* fully see-through */
border-color: currentColor; /* text color jo bhi ho, wahi use hoga */
```

Practical Tools: `color` , `background-color` , `border-color` , `outline-color` , `box-shadow` , aur gradients — sab jagah in sab color formats mein se koi bhi use kar sakte ho.

Sizing Units — Numbers & Percentages

Unitless Numbers

```
line-height: 1.5; /* font-size ka 150% — RECOMMENDED */
opacity: 0.5;
transform: scale(1.2);
```

line-height Best Practice: Unit ke bagair likho (jaise `1.5` , px nahi). Ye font-size ke relative rehta hai — agar font-size inherit ho kar change ho, line-height bhi proportionally adjust ho jayega.

Percentages — Context Pe Depend Karta Hai

Property	% Kis Ke Relative Hai
<code>width</code>	Parent element ki available width
<code>margin</code> / <code>padding</code> (kisi bhi side)	Hamesha parent ki width (height nahi, chahe top/bottom ho)
<code>transform: translateX()</code>	Khud us element ki apni size

Confusing Trap: `margin-top: 50%` aur `padding-left: 50%` dono parent ki **width** se calculate hote hain — height se nahi! Ye bohat log confuse karta hai.

Absolute Lengths — Print Ke Liye Best

Unit	Equivalent
<code>px</code>	1/96th of 1 inch
<code>cm</code> / <code>mm</code>	Physical centimeters/millimeters — print stylesheets ke liye reliable
<code>in</code> / <code>pt</code> / <code>pc</code>	Inches, points, picas

Physical units (cm, in) screen par accurate nahi hote (pixel density variations ki wajah se) — best print ke liye.

Sizing Units — Relative Lengths

Font-Size-Relative Units

Unit	Relative To
<code>em</code>	Parent ke computed font-size (1.5em = parent se 50% bada)
<code>rem</code>	Root (<code><html></code>) ka font-size — default 16px, consistent hamesha
<code>ch</code>	Font ke "0" character ki average width — line-length control ke liye perfect
<code>ex</code>	Lowercase "x" ki height (approx)
<code>lh</code>	Element ki line-height

```
max-width: 60ch; /* paragraph readability – 60 characters se zyada nahi */
```

User Preference Respect Karo: Text sizing ke liye `px` ki jagah `em` / `rem` use karo — agar user ne apne OS/browser mein bada default font-size set kar rakha hai, ye units us preference ko honor karenge, px nahi karega.

Viewport-Relative Units

Unit	Relative To
<code>vw</code>	Viewport width ka 1%
<code>vh</code>	Viewport height ka 1%
<code>vmin</code> / <code>vmax</code>	Viewport ki chhoti/badi dimension ka 1%

Mobile Browser Gotcha: Mobile browsers UI elements (address bar) show/hide karte rehte hain, jisse viewport ka visible area badalta hai lekin `vh` nahi badalta. Fix ke liye `dvh` (dynamic viewport height) use karo — ye actual visible area track karta hai.

Container Query Units

```
font-size: 5cqi; /* container ke inline-size ka 5% */
```

`cqw` , `cqh` , `cqi` , `cqb` — parent viewport ki jagah parent **container** ke dimensions ke relative sizing (container queries ke andar use hote hain).

Layout — Overview

display property 2 kaam karta hai: (1) element khud **inline ya block** behave karega, (2) uske **children** kis tarah layout honge.

Display	Behaviour
inline	Words jaisa — explicit width/height set nahi kar sakte, surrounding whitespace preserve hota hai
block	Naya line banata hai, full inline width tak expand, margin sab sides par respect hota hai
flex	Children flex items ban jate hain — one-dimensional layout control
grid	Children grid items ban jate hain — two-dimensional (rows + columns) control

Flow Layout Ke Alternative Methods

inline-block

Text ke sath inline flow karta hai, lekin margin/padding top-bottom bhi respect hota hai — best of both.

Floats

float: left/right — text ko image ke around wrap karwata hai (newspaper style). Clearfix ya **display: flow-root** se side-effects fix karo.

Multicolumn Layout

```
.countries {
  column-count: 2;
  column-gap: 1em;
}
/* ya responsive: */
.countries { column-width: 260px; } /* auto columns jitne fit hon */
```

Positioning Quick Reference

Value	Behaviour
static	Default — normal document flow
relative	Apni jagah se nudge; children ke liye "containing block" ban jata hai
absolute	Flow se completely nikal jata hai, nearest relative parent ke against position
fixed	Root <html> ke against, scroll se unaffected
sticky	relative + fixed ka hybrid — scroll tak normal, phir anchor ho jata hai

Flexbox — Axes & Growth

Main Axis vs Cross Axis

Main axis = `flex-direction` ki direction (row ya column). Items ek group ki tarah main axis par move karte hain. **Cross axis** = main axis ke perpendicular.

```
.container {
  display: flex;
  flex-direction: row; /* row | row-reverse | column | column-reverse */
  flex-wrap: wrap;      /* default: nowrap (overflow ho sakta hai) */
}
```

Accessibility Warning: `row-reverse`, `column-reverse`, aur `order` sirf **visual** order badalte hain — DOM order same rehta hai. Keyboard navigation aur screen readers DOM order follow karte hain, jo disconnect create kar sakta hai. Sirf reorder karo agar HTML mein bhi genuinely different order hona chahiye.

flex Shorthand — Grow, Shrink, Basis

Value	Kya Karta Hai
<code>flex: initial</code>	grow:0, shrink:1, basis:auto — items apni content size par rukte hain
<code>flex: auto</code>	grow:1, shrink:1, basis:auto — items grow karte hain, content size proportional
<code>flex: 1</code>	grow:1, shrink:1, basis:0 — sab items EQUAL size (content ignore)
<code>flex: none</code>	Grow/shrink kuch nahi — rigid items

```
.item1 { flex: 1; } /* 1 part */
.item2 { flex: 2; } /* 2 parts – double space */
.item3 { flex: 3; } /* 3 parts */
```

Space 6 hisson mein baant kar distribute hoga (1+2+3), har item apna proportion leta hai.

Flexbox — Alignment Properties

Property	Axis	Kya Karta Hai
<code>justify-content</code>	Main	Extra space distribute karta hai items ke beech
<code>align-content</code>	Cross	Wrapped flex LINES ke beech space distribute karta hai
<code>align-items</code>	Cross	Sab items ko group ki tarah cross axis par align karta hai
<code>align-self</code>	Cross	Ek individual item ko cross axis par align karta hai

Naming Pattern: Main axis ki properties `justify-` se shuru hoti hain, cross axis ki `align-` se. Koi `justify-self` flexbox mein nahi hota kyunki main axis par items ek group ki tarah treat hote hain — individual split nahi ho sakta (auto margins se workaround milta hai).

Perfect Centering — Classic Pattern

```
.container {  
  display: flex;  
  justify-content: center; /* horizontal */  
  align-items: center;    /* vertical */  
}
```

Auto Margin Trick — Group Split Karna

```
.last-item { margin-left: auto; } /* isko dhakka de kar right pe le jayega */
```

Auto margin apni direction ka **saara available space** absorb kar leta hai — is se items ke groups ko split kar sakte ho bina extra wrapper ke.

Note: `justify-content` aur `align-content` sirf tab kaam karte hain jab container mein **extra space** ho — agar items already puri width le rahe hon, koi effect nahi hoga.

Grid — Terminology & Track Sizing

Grid **two-dimensional** layout ke liye hai (rows + columns saath) — flexbox sirf ek dimension handle karta hai.

Term	Matlab
Grid Line	Horizontal/vertical divider, number 1 se start
Grid Track	Do lines ke beech ka space (row ya column)
Grid Cell	Sabse chhota unit — ek row aur ek column ka intersection
Grid Area	Multiple cells jo saath mile hon (item ko span karwa kar banta hai)
Gap	Tracks ke beech gutter — content nahi rakh sakte, lekin span kar sakte ho

```
.container {
  display: grid;
  grid-template-columns: 5em 100px 30%;
  grid-template-rows: 200px auto;
  gap: 10px;
}
```

The fr Unit — Flexible Space Sharing

```
grid-template-columns: 1fr 1fr 1fr;      /* 3 equal columns */
grid-template-columns: 200px 1fr;        /* fixed + remaining space */
```

fr = fraction of remaining space, laid out **after** content ki size calculate ho jaye — **flex: auto** jaisa behaviour.

minmax() aur repeat()

```
grid-template-columns: repeat(12, minmax(0, 1fr)); /* 12-column grid, sab equal */
```

minmax(0, 1fr) Kyun Zaroori: Agar sirf **1fr** use karo, tracks content ke min-content se chhote nahi ho sakte — overflow ho sakta hai. **minmax(0, 1fr)** minimum ko zero force karke sahi equal-width guarantee karta hai.

auto-fill vs auto-fit

```
grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
```

Bina media query ke responsive grid! Jitne 200px tracks fit ho sakein utne banayega. **auto-fill** empty tracks chhod deta hai; **auto-fit** unhe collapse kar ke available items ko stretch kar deta hai.

Grid — Placement & Subgrid

Line-Based Placement

```
.item {
  grid-column: 1 / 4; /* line 1 se line 4 tak (shorthand) */
  grid-row: 2 / 4;
}
/* ya span keyword se: */
.item { grid-column: span 2; } /* 2 tracks span karega */
```

Named Grid Lines

```
grid-template-columns: [main-start] 1fr [content-start] 2fr [main-end];
.sidebar { grid-column: main-start / content-start; }
```

grid-template-areas — Visual Layout

```
header { grid-area: header; }
.sidebar { grid-area: sidebar; }

.container {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  grid-template-areas:
    "header header header header"
    "sidebar content content content"
    "sidebar footer footer footer";
}
```

Bohat Powerful: CSS mein hi layout ka "visual shape" dikh jata hai. Empty cell ke liye `.` use karo. Media queries ke andar poori `grid-template-areas` redefine karke layout completely reshape kar sakte ho.

Subgrid

```
.nested-grid {
  display: grid;
  grid-column: span 2;
  grid-template-columns: subgrid; /* parent ki tracks inherit karta hai */
}
```

Nested grid parent ki track sizing, line names, aur gap use karta hai — items ko parent grid ke sath perfectly align karne ke liye (jaise cards ke andar captions sab align rahen).

Grid Alignment (Same as Flexbox)

Property	Applies To
<code>justify-items</code> / <code>align-items</code>	Container par — sab items ko unke area ke andar move karta hai

